

Pyramidal Flow Matching

Efficient video generation with spatial and temporal pyramidal flow matching

Paper by Yang Jin, Zhicheng Sun, Ningyuan Li, Kun Xu, Kun
Huang, Yang Song, Yadong Mu, Zhouchen Lin

Marcel Sigl

Advanced Machine Learning · 2026

Outline

01

Motivation + preliminaries

Why full-resolution video is expensive, and what flow matching makes possible. §1–2

02

S

03

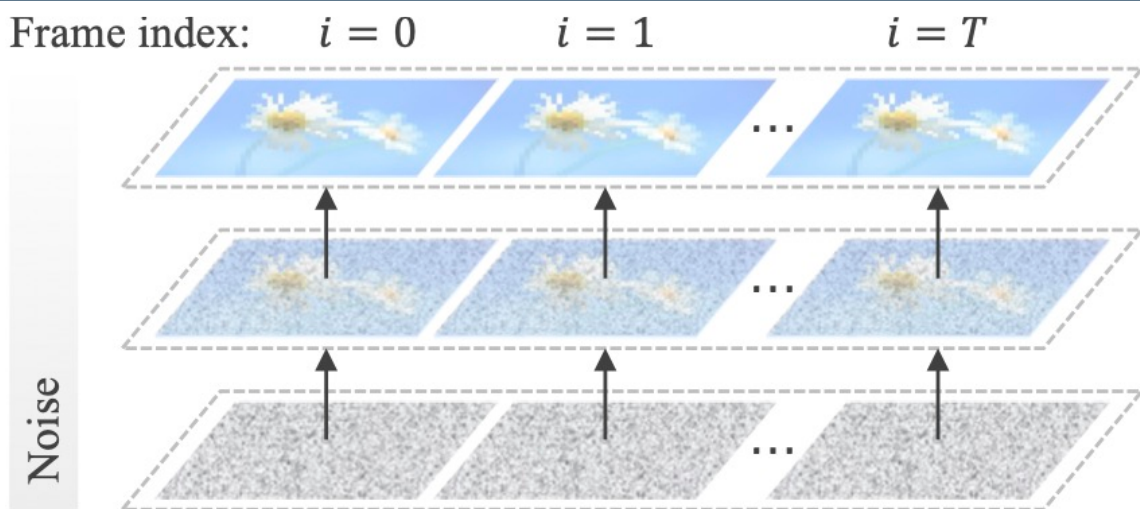
Video generation

Compress distant history and preserve causality across frame units. §4

04

E

Motivation



(a) Video diffusion model like Sora (OpenAI, 2024)

Full-resolution diffusion

pays for fine spatial grids even when the latent is mostly noise.

Pyram
increa

Prior Approaches

ONE FULL-RESOLUTION PATH

Every model call sees the largest latent grid

768p

×

240 frames

×

many solver steps

- High attention and memory cost from the first noisy state
- Limits batch size, duration, or output resolution

SEPARATE PATHS

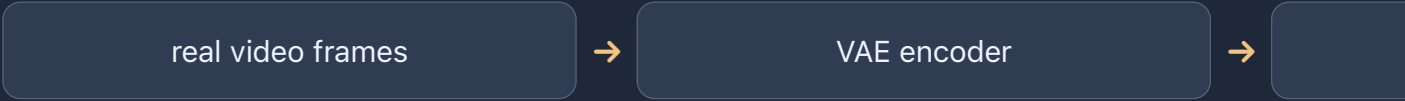
Generate

- Multiple
- Weights

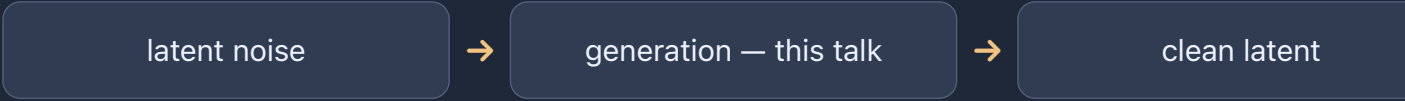
Pyramidal Flow seeks one end-to-end path that uses small grids early

Latent Video Generation

TRAINING · HOW THE MODEL SEES VIDEOS



GENERATION · NO INPUT FRAMES — STARTS FROM NOISE



Compression: 8× height · 8× width · 8× time

Causal: the first frame is encoded alone
valid one-frame video

The VAE is a separate model—trained once, then frozen. Everything in this talk happens in

Flow Matching vs. Diffusion

Diffusion

defines a stochastic corruption process first, then learns to reverse it—curved probability paths that need many steps.

Like diffusion: iterative generation from noise

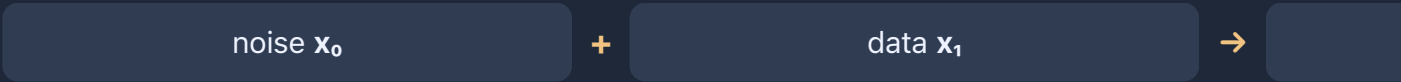
Flow

chooses a path from noise to data

Unlike diffusion: the path is designed to be straight

Flow Matching

TRAINING · ONE RANDOM TIME



MATH

$$x_t = (1-t)x_0 + tx_1$$
$$\text{target} = x_1 - x_0$$

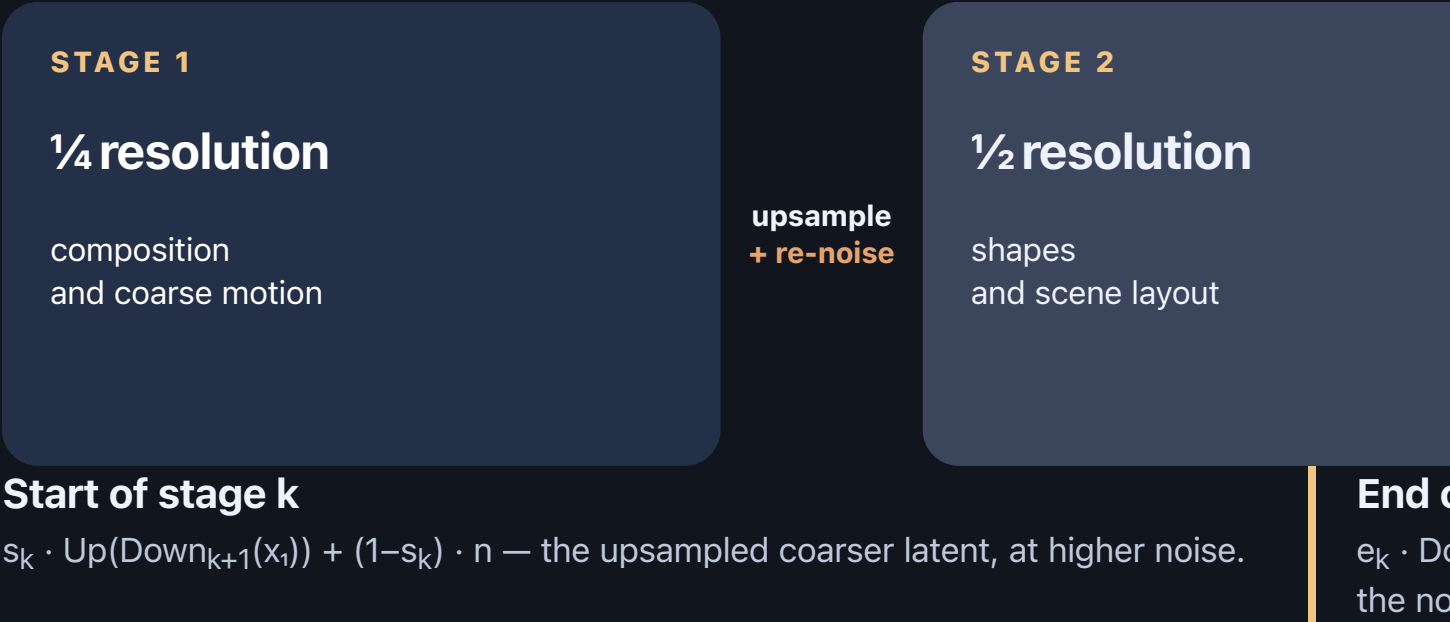
```
t = torch.rand(batch, 1, device=device)
x_t = (1 - t) * x_0 + t * x_1
target = x_1 - x_0
loss = (dit(x_t, t) - target).abs().mean()
```

INFERENCE · SOLVE THE LEARNED ODE



Nothing fixes the path’s endpoints. A segment can start at one resolution and end at another.

The Spatial Pyramid



Each stage is a straight flow between two noisy versions of the *same* video. The arrows are labeled with the noise statistics.

Attention Cost

Tokens per frame

Quarter resolution means a quarter of the width and a quarter of the height—only 1/16 of the tokens.

Most solver steps run on the small grids; the full grid is only touched at the end. One caveat: the cost of the full grid is \$4.

Attention

Attention is roughly

Stages in Practice



160 × 96

global color and layout



320 × 192

objects and motion stabiliz

The coarse stage already chooses the scene; later stages spend compute on increasingly

Stage Transitions

1 · finish a stage

Integrate the learned flow on the current grid.



2 · upsample

Copying values creates blockwise cor

```
latents = F.interpolate(
    latents, size=(height, width), mode="nearest"
)
noise = self.sample_block_noise(
    bs, ch, temp, height, width
)
latents = alpha * latents + beta * noise
```

Pyramidal Training

clean video



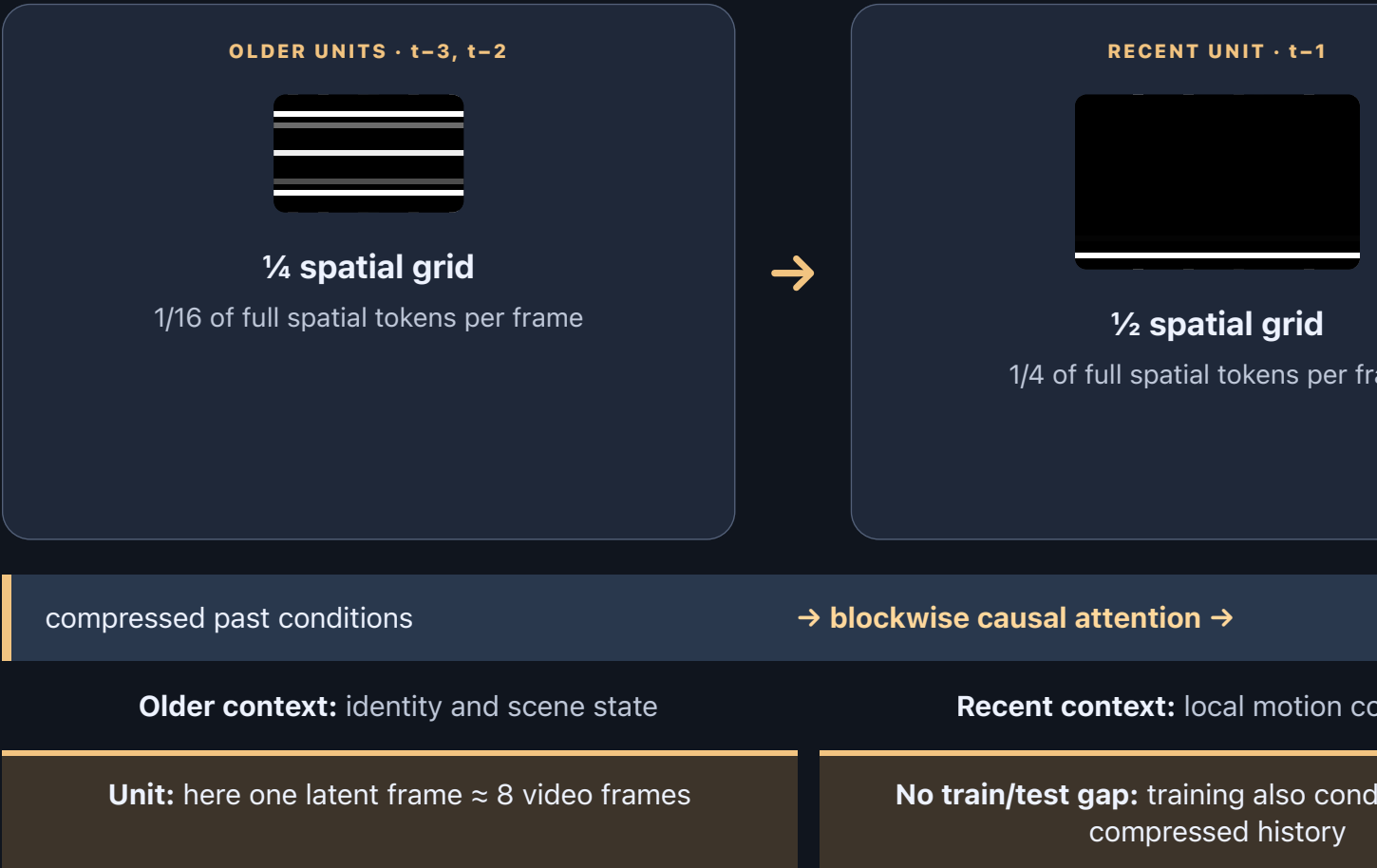
VAE latent pyramid



sample stage + time

```
noisy_latents = (  
    ratios * start_point  
    + (1 - ratios) * end_point  
)  
target = start_point - end_point  
prediction = dit(noisy_latents, timesteps)  
loss = (prediction - target).square().mean()
```

The Temporal Pyramid



Architecture

PROMPT · FROZEN, PRETRAINED

Text encoder

token embeddings + pooled embedding

PIXELS · SEPARATE MODEL, FROZEN

Causal video VAE

encode frames → latent pyramid

PAST UNITS · NO PARAMETERS

Temporal pyramid

coarse old context + detailed recent context

conditions
→

ONE SHARED NETWORK · TRAINED IN THE WHEEL

Diffusion Transformer

inputs: current noisy latent, past conditions

output: velocity field

Only the DiT is trained. The VAE and text encoder are frozen separate models; the scheduler is pluggable

The Inference Loop

`generate_one_unit()` · CORE LOOP

```
for stage in range(len(stages)):
    scheduler.set_timesteps(steps[stage], stage)

    if stage > 0:
        latents = F.interpolate(latents, mode="nearest")
        noise = sample_block_noise(latents.shape)
        latents = alpha * latents + beta * noise

    for t in scheduler.timesteps:
        model_input = past_conditions[stage] + [latents]
        velocity = dit(model_input, t, text_embeddings)
        latents = scheduler.step(velocity, t, latents).prev_sample
```

Pyramid-Flow/pyramid_dit/pyramid_dit_for_video_gen_pipeline.py · lines 927–1031 · own addition: expo

Convergence Ablation

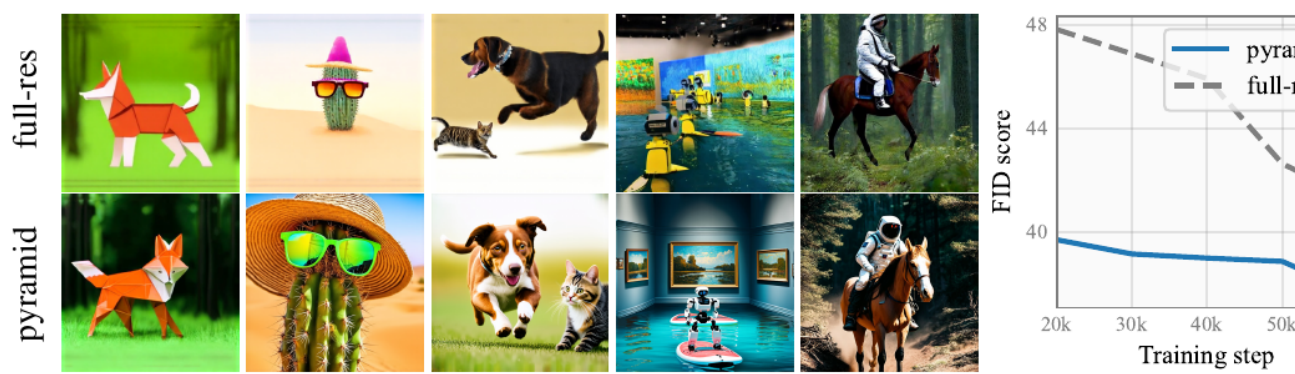


Figure 7: Ablation study of spatial pyramid at 50k image training step. On the right is a quantitative comparison of the FID results, where our method achieves almost three times the convergence speed.

VBench Results

open-source data only

20.7k A100 GPU hours

5–10 s clips

System	Total	Quality	Semantic
Gen-3 Alpha commercial	82.32	84.11	75.17
VideoCrafter2	80.44	82.20	73.42
T2V-Turbo	81.01	82.57	74.76
Pyramid Flow	81.72	84.74	69.62

Conclusion

What the pyramid gets right

- Compute follows information content.
- One DiT shares knowledge across scales.
- Spatial and temporal savings compound.
- Text-to-video and image-to-video fit naturally.

What

- Au
- Co
- St
- Qu

PYRAMIDAL FLOW MATCHING

Use coarse grids while un
Increase resolution as struct

Live demo → [question](#)

PYRAMID FLOW

DEMO

Local Gradio interface